

Literature Review Sheet

Thesis: An Empirical Evaluation and Benchmark of AI-Generated Vulnerabilities in Code

Name: Eyad Hassan Abdelfatah

Models used in my pipeline: GPT-4o-mini | Claude 3.5 Sonnet | Llama 3.1 8B Instruct (open-source)

CWEs in scope: CWE-89 SQL Injection | CWE-78 OS Command Injection | CWE-22 Path Traversal **Languages:** Python (Java/JavaScript planned)

Paper	Main Features	Method	Key Results	Gap Addressed	Gap Left	Link	Comment
Sabra, Schmitt & Tyler (2025), Sonar (arXiv 2508.14727)	5 LLMs (Claude Sonnet 4/3.7, GPT-4o, Llama 3.2 90B, OpenCoder-8B); 4,442 Java tasks; pass-rate vs. quality/security.	3 Java benchmarks; Pass@1 on 2; SonarQube ~550 rules (bugs/vulns/smells by severity).	No pass-quality correlation; best model 77% pass, still 2.11 issues/task; ~2% vulns / 90–93% smells across all models; newer Claude had <i>more</i> severe issues despite higher pass rate.	Refutes “better pass rate = safer code”, using frontier commercial models.	Java only; generic rules, no CWE/CVE mapping, no exploit verification; vulns only ~2% of findings.	https://arxiv.org/abs/2508.14727	Quality study, not vulnerability-focused. Useful only for the “pass rate $\neq$ security” motivating fact.
Shahid, Ahmed & Ranjan (2025), LLM-CSEC (arXiv 2511.18966)	10 LLMs on C/C++; plain (CG) vs. “secure” (SCG) persona — does security prompting help?	84 CWE-based prompts; 3 SAST tools in parallel (CodeQL, Snyk, CodeShield); CWE→CVE mapping via NVD.	Every model produced critical vulns regardless of persona; “secure” persona sometimes worse; 3 tools barely agreed (33/10/1 CWEs found, 0 common).	First vuln-first benchmark with real CWE→CVE mapping; proves single-tool SAST is unreliable.	Static only, no exploit verification; CVE mapping is historical, not this-snippet; C/C++ only.	https://arxiv.org/html/2511.18966v1	Strongest motivator for exploitability as a gap — but CVE mapping is still indirect, not a direct test.
Li et al. (2025), SafeGenBench, ByteDance (arXiv 2506.05692)	558 test cases, 44 CWE types, 12 languages; dual-judge (SAST + LLM-as-judge) detection.	Semgrep + DeepSeek-R1 judge, independent 0/1 scoring; 13 models; zero-shot / safety-instruction / few-shot.	37.44% avg. zero-shot accuracy; safety instructions +20pts; judges agreed only 61% of the time (2.6% flagged by both).	Quantifies (not just asserts) single-method detection is unreliable; broadest coverage reviewed.	Detection-only (own stated limit); single-function only; 1 SAST tool + 1 LLM judge; Chinese-language prompts.	https://arxiv.org/abs/2506.05692	Closest template to my pipeline’s detection stage — explicitly admits detection $\neq$ proof of exploitability.
Yoo & Kim (2025), Automated Code Generation Security (Tehnicki Glasnik 19(4))	20 ChatGPT C programs vs. 20 human-written; hybrid static+dynamic security analysis.	Ghidra/IDA/Objdump + Valgrind + Frida; OWASP Top 10 / STRIDE mapping; 2 samples with full attack scenarios.	AI code +6.4% more vulns; tools caught only 46.7% of AI vulns vs. 66.4% human; 100%-accuracy XOR key recovery demo.	First to go beyond CWE-tagging into worked attack scenarios (key recovery, DoS, MITM).	Only 2 hand-picked samples, no scripted/automated PoC; 20 programs total; ChatGPT/C only.	DOI link	Closest precedent for exploit confirmation — but hand-crafted, not a scaled pipeline. My thesis automates this.
“Broken by Default” (2026, arXiv 2604.05292)	7 LLMs (GPT-4o, Gemini 2.5 Flash, others); 3,500 artifacts, 500 prompts; 5 CWE categories incl. CWE-89/22/78.	Z3 SMT solver → formal proof of exploitability + witness value; subset validated via AddressSanitizer (6/7 crashes confirmed).	55.8% mean vuln rate; no model scored above D; 1,055 vulns formally proven; tools missed 97.8% of them.	First paper found that rigorously confirms exploitability (not just detects) in fresh LLM code, at scale.	C/C++-focused (Python limited, no Java/JS); proof method fits memory-safety, not functional injection exploits; no cross-model PoC comparison.	https://arxiv.org/html/2604.05292v1	Concurrent work — closes part of the “nobody confirms exploitability” gap. Narrowed my own gap statement accordingly.
Zhang et al. (2025), PoCGen (arXiv 2506.04962)	Generates <i>working</i> PoC exploits for known npm/JS CVEs; 5 vuln types (path traversal, cmd injection, etc.).	LLM + CodeQL (taint paths) + dynamic execution, iterative refine-on-failure loop.	77% success on SecBench.js (vs. 32% prior tool); 5/6 real exploits accepted into CVE reports; \$0.02/exploit.	Proves LLM+static+dynamic loop reliably produces <i>working</i> exploits, not just plausible code.	Different question: exploits already-known CVEs, not vulns an LLM freshly introduces.	https://arxiv.org/abs/2506.04962	Not competing with my thesis, but strongest methodology reference if I extend to JavaScript.

Paper	Main Features	Method	Key Results	Gap Addressed	Gap Left	Link	Comment
Pearce, Ahmad, Tan, Dolan-Gavitt & Karri (2022), “Asleep at the Keyboard?” (IEEE S&P, arXiv 2108.09293)	GitHub Copilot; 1,689 programs across 89 scenarios targeting MITRE CWE Top-25.	Scenario-based prompts per CWE; manual/scripted review of completions; no exploit confirmation.	~40% of completions vulnerable; risk varies by CWE and application domain.	First systematic “does Copilot suggest insecure code” study at scale, across many CWEs.	Detection only (manual/pattern review); no automated exploit confirmation; predates today’s frontier models.	https://arxiv.org/abs/2108.09293	The foundational baseline nearly every later paper in this space cites. My exploit-confirmation stage is the piece missing here.
Siddiq & Santos (2022), SecurityEval Dataset (MSR4P&S’22)	130 hand-curated prompts spanning 75 CWE types, built as ground truth for evaluating ML-based code generation.	Manual dataset construction, mined from real CWE/CVE examples; model-agnostic prompt set.	Established as a reusable benchmark; adopted by several later studies as their prompt source.	Provides a reliable, CWE-labeled prompt set, addressing the lack of standardized ground truth in earlier ad hoc studies.	A static prompt dataset only – no detection, exploit confirmation, or model evaluation of its own.	ACM DL link	Useful precedent for my own hand-built calibration set; stops at prompt design where mine continues into detection and exploit confirmation.
Khoury, Avila, Brunelle & Camara (2023), “How Secure is Code Generated by ChatGPT?” (arXiv 2304.09655)	21 small ChatGPT-generated programs across 5 languages, each targeting a common vulnerability class.	Manual security review; follow-up prompts asking ChatGPT to find and fix its own flaws.	Code insecure by default in most cases; ChatGPT fixed flaws only when explicitly asked.	One of the earliest studies to show insecure-by-default behavior in conversational (chat-style) code generation.	Only 21 programs, single model, purely manual review; no automated detection or exploit confirmation.	https://arxiv.org/abs/2304.09655	Small and early, but frequently cited as the first ChatGPT-specific security study.
Fu, Liang, Tahir, Li, Shahin, Yu & Chen (2025), Copilot GitHub Study (TOSEM, arXiv 2310.02059)	Real, already-committed Copilot completions mined from GitHub projects (not synthetic prompts); 733 flagged snippets, 43 CWE categories.	CodeQL-based static detection on real-world repositories; tests whether Copilot Chat can fix flagged issues given the static-analysis warning.	29.5% of Python and 24.2% of JavaScript snippets affected; Copilot Chat fixed up to 55.5% of flagged issues when prompted with the warning.	Moves past synthetic single-function prompts to real production code actually merged into projects.	Detection only (CodeQL); no exploit confirmation. Fixing is evaluated, exploitability is not.	https://arxiv.org/abs/2310.02059	Strongest “real-world code” precedent found – complements the synthetic-prompt studies already in this table.
Hamer, d’Amorim & Williams (2024), “Just another copy and paste?” (arXiv 2403.15600)	Compares vulnerability rates between ChatGPT-generated code and matched StackOverflow answers for the same tasks.	Paired-task design; identical static security analysis applied to both code sources.	ChatGPT code was not meaningfully safer than StackOverflow snippets it may have been trained on; similar vulnerability patterns in both.	Tests the “LLMs just regurgitate StackOverflow” hypothesis directly, with an apples-to-apples baseline.	Detection only; no exploit confirmation; single-model (ChatGPT) comparison.	https://arxiv.org/abs/2403.15600	Useful baseline for arguing AI-generated code is not obviously better than the insecure code it may have learned from.
Hajipour, Hassler, Holz, Schönherr & Fritz (2023), CodeLMSec Benchmark (arXiv 2302.04012)	Automatically generates prompts that reliably elicit vulnerable completions from black-box code LLMs.	Few-shot prompt optimization targeting known vulnerability classes; benchmark built from the resulting adversarial prompts.	Shows current code LLMs can be reliably steered into vulnerable completions across multiple CWE categories.	First systematic, automated approach to finding an LLM’s vulnerable “blind spots,” rather than relying on fixed hand-written prompts.	Detection-focused; adversarial prompt design, not exploit confirmation of the resulting code.	https://arxiv.org/abs/2302.04012	Different angle (adversarial prompt discovery vs. neutral prompts + exploit confirmation), but a useful methodological contrast.
Bhatt et al. (2023-2025), Purple Llama CyberSecEval, Meta (arXiv 2312.04724 + v2/v3)	Industry-scale benchmark suite (Llama 2, Code Llama, GPT family); insecure-code generation plus willingness to assist cyberattacks.	Automated static analysis across many CWEs and languages; separate cyberattack-helpfulness test suite.	More capable models were, if anything, more likely to suggest insecure code; broad multi-language, multi-model coverage.	Largest industry benchmark to date; establishes a security-risk baseline across an entire model family.	Static detection only, no exploit confirmation; industry benchmark, not a traditionally peer-reviewed paper.	https://arxiv.org/abs/2312.04724	Useful “industry baseline” contrast to the academic papers in this table – shows the problem is recognized at the frontier-lab level too.

Paper	Main Features	Method	Key Results	Gap Addressed	Gap Left	Link	Comment
Ding, Fu, Ibrahim, Sitawarin, Chen, Alomair, Wagner, Ray & Chen (2024), PRIMEVUL (arXiv 2403.18624)	Audits existing vulnerability-detection benchmarks for code LMs; finds widespread mislabeling, duplication, unrealistic splits. Introduces PRIMEVUL, a relabeled benchmark with chronological splits.	Manual and automated re-labeling of prior datasets; evaluates 7 models, incl. GPT-4, on old vs. new benchmarks.	Models scoring 68% F1 on prior (flawed) benchmarks scored just 3% on PRIMEVUL – most prior “good” results were measurement artifacts.	Shows dataset quality, not just model capability, is a major open problem in this field.	Still detection-only; does not touch exploit confirmation – but strongly motivates why ground-truth quality matters.	https://arxiv.org/abs/2403.18624	Important methodological warning – directly supports why I built and validated my own ground truth instead of reusing an existing dataset.
Zhang, Yang, Su, Weyssow, Nguyen, Bui, Kang, Li, Ouh, Shar & Lo (2025), Multi-Language SVD (arXiv 2503.01449)	Datasets of vulnerable functions in Python, Java, and JavaScript – the same three languages as my thesis; 5 open-source LLMs evaluated as detectors.	Prompt engineering and fine-tuning tested per language; data-balancing and ensemble methods proposed as improvements.	Vulnerability detection remains hard for LLMs across all three languages; balancing/ensembling help but do not close the gap.	Directly matches my language scope (Python/Java/JS) in one consistent benchmark, rare among papers reviewed.	Detection only; LLMs used as detectors, not as the code generators under test; no exploit confirmation.	https://arxiv.org/abs/2503.01449	Closest language-scope match to my thesis of any paper found – good direct comparison point.
Peng, Cui, Huang, Yang & Ray (2025), CWEval (arXiv 2501.08200)	Evaluates generated code on functional correctness and security together, via outcome-driven (execution-based) test oracles; CWEVAL-BENCH: 119 tasks, 31 CWE types, multiple languages.	Executes generated code against both functionality tests and security oracles; compares against static-analysis baselines.	Reveals a real gap between functional correctness and security: many functionally-correct completions are still exploitable; execution-based oracles catch cases static analysis misses.	Moves detection from pattern-matching to actually running the code and checking outcomes – conceptually the closest paper to my own exploit-confirmation approach.	Oracles confirm the flaw is triggerable via execution, not that it forms a full attacker-controlled exploit path (e.g. proving data exfiltration).	https://arxiv.org/abs/2501.08200	The closest existing work to my own methodology; worth directly contrasting my harness design against their oracle design.
Shukla, Joshi & Syed (2025), Security Degradation in Iterative AI Code Generation (IEEE-ISTAS, arXiv 2506.11022)	400 code samples across 40 rounds of iterative refinement, using 4 different prompting strategies.	Repeated “improve this code” prompting cycles; static vulnerability scanning after each round.	Critical vulnerabilities increased 37.6% after just 5 iterations – refinement made code less secure, not more.	Directly relevant motivating finding: naive iterative LLM use can actively worsen security over time.	Detection only, static analysis; no exploit confirmation of whether the added vulnerabilities are actually exploitable.	https://arxiv.org/abs/2506.11022	Directly informs a methodological choice in my own pipeline – generating once per prompt rather than iterating, partly because of this finding.
Cotroneo, Improtà & Liguori (2025), Human-Written vs. AI-Generated Code (arXiv 2508.21634)	Over 500,000 human-written and AI-generated (ChatGPT, DeepSeek-Coder, Qwen-Coder) code samples, Python and Java.	Large-scale static analysis for defects, vulnerabilities, and complexity metrics across both code sources.	AI-generated code is simpler/more repetitive but shows a higher prevalence of serious security vulnerabilities than human code.	Largest-scale human-vs-AI vulnerability comparison found, and multi-model (3 code LLMs).	Detection only, no exploit confirmation; does not test whether the found vulnerabilities are actually reachable/exploitable.	https://arxiv.org/abs/2508.21634	Strong large-scale evidence for the underlying premise of my thesis – that AI code has a real, elevated vulnerability rate worth studying further.
Mousavi, Islam, Babar, Abuadbbba & Moore (2026), AI Code Assistants & Security API Usage (arXiv 2607.11348)	User study: 44 professional developers completing security-relevant coding tasks with and without GitHub Copilot.	Controlled tasks measuring correct security-API usage, functional correctness, and developer awareness of introduced flaws.	Copilot improved functional correctness but did not significantly improve secure API usage; developers often missed vulnerabilities and rarely flagged security concerns.	The only human-in-the-loop study found – shows the risk is not just “does the model write bad code” but “do developers catch it.”	No exploit confirmation; qualitative/task-based, not a large-scale automated benchmark.	https://arxiv.org/abs/2607.11348	Different methodology entirely (human study vs. automated pipeline), but frames why automated exploit confirmation like mine matters – developers aren’t reliably catching these issues themselves.

Gap Analysis Matrix

Paper	AI Models Tested	Detects Vulnerabilities?	Confirms Exploitability?	Programming Languages	Vulnerability Types (CWE)	What Is Missing
Sabra et al. (Sonar)	5 models: Claude Sonnet 4, Claude 3.7 Sonnet, GPT-4o, Llama 3.2 90B, OpenCoder-8B	Yes	No	Java	Not mapped to specific CWEs; uses generic static-analysis rules only	Uses one static-analysis tool on one language, and never tests whether a flagged vulnerability can actually be exploited.
LLM-CSEC	10 models, including CodeLlama	Yes	No	C, C++	Mapped to historical CVE records, not tied to a fixed CWE list	Shows that a type of vulnerability has caused real damage in the past, but never tests whether this specific generated snippet is exploitable.
SafeGenBench	13 models: GPT-4o, o1, o3, Claude 3.5, Claude 3.7, Gemini 2.5 Pro, DeepSeek, Qwen3, Llama 4	Yes	No (stated as a limitation by the authors)	12 programming languages	44 CWE types across 8 vulnerability categories	Detects vulnerabilities using two independent methods, but never confirms that a flagged vulnerability is actually exploitable.
Yoo & Kim	1 model: ChatGPT	Yes	Partial — only 2 examples	C	Not organized by CWE; based on manually chosen attack scenarios (key recovery, denial of service)	Confirms exploitability, but only for 2 hand-picked examples out of many findings; not a repeatable, automated process.
Broken by Default	7 models, including GPT-4o and Gemini 2.5 Flash	Yes	Yes	C, C++ (Python covered only in a limited part of the study; no Java or JavaScript)	5 categories, one of which groups SQL Injection (CWE-89), OS Command Injection (CWE-78), and Path Traversal (CWE-22) together without separating them	Confirms exploitability with a mathematical proof method suited to memory-corruption bugs, not the functional exploitation (e.g. stealing data via SQL Injection) needed for the vulnerabilities this thesis studies. Also does not cover Java or JavaScript, or compare models using one consistent method.
PoCGen	Uses an LLM to help write exploits; does not benchmark code-generation models	Not applicable	Yes	JavaScript	Path Traversal, Command Injection, Prototype Pollution, Code Injection, ReDoS	Builds working exploits for vulnerabilities that are already known and publicly reported; does not study whether an AI model introduces new vulnerabilities when generating code.
Asleep at the Keyboard (Pearce et al.)	1 model: GitHub Copilot	Yes	No	Python, C	Not organized by a fixed list; scenarios drawn from MITRE's CWE Top-25	Detects vulnerable completions via manual/pattern review only; never tests whether the flagged code is actually exploitable, and predates today's frontier models.
SecurityEval Dataset (Siddiq & Santos)	Not applicable (a prompt dataset, not a model evaluation)	Not applicable	Not applicable	Python	75 CWE types	A reusable, CWE-labeled prompt set only; performs no detection, exploit confirmation, or model evaluation of its own.
How Secure is Code Generated by ChatGPT? (Khoury et al.)	1 model: ChatGPT	Yes	No	5 languages (not individually named in the abstract)	Not organized by CWE; targets common vulnerability classes informally	Only 21 hand-reviewed programs from a single model; no automated detection or exploit confirmation.
Copilot GitHub Study (Fu et al.)	3 tools: GitHub Copilot, Amazon CodeWhisperer, Codeium	Yes	No	Python, JavaScript (plus others across 733 snippets)	43 CWE categories	Detects real, already-committed code with CodeQL and tests whether Copilot Chat can fix it, but never tests whether the flagged code is exploitable.

Paper	AI Models Tested	Detects Vulnerabilities?	Confirms Exploitability?	Programming Languages	Vulnerability Types (CWE)	What Is Missing
Just Another Copy and Paste? (Hamer et al.)	1 model: ChatGPT	Yes	No	Java	Not organized by CWE	Compares vulnerability rates to StackOverflow answers but never confirms exploitability in either source; single-model, detection-only.
CodeLMSec Benchmark (Hajipour et al.)	Black-box code LLMs (not individually named in the abstract)	Yes	No	Not specified by name in the abstract	Multiple CWE categories (adversarially targeted, not a fixed list)	Automatically finds prompts that elicit vulnerable completions but never confirms whether the resulting code is actually exploitable.
Purple Llama CyberSecEval (Bhatt et al., Meta)	Llama 2, Code Llama, GPT family (multiple models/versions)	Yes	No	Multiple, incl. C, C++, Python, Java, JavaScript	Broad range of categories via static-analysis rules	Large industry static-analysis benchmark; never confirms exploitability, and is an industry report rather than peer-reviewed research.
PRIMEVUL (Ding et al.)	7 models, including GPT-4	Yes (evaluated as detectors)	Not applicable	C, C++, Python	Multiple (inherited from underlying vulnerability datasets)	Focuses on fixing dataset-quality problems in detection benchmarks; does not address exploit confirmation at all.
Multi-Language SVD (Zhang et al.)	5 open-source LLMs	Yes (evaluated as detectors)	Not applicable	Python, Java, JavaScript	Multiple (dataset-derived, not a fixed enumerated list)	Evaluates LLMs as vulnerability detectors across three languages but never generates code or confirms exploitability.
CWEval (Peng et al.)	Multiple leading LLMs (not individually named in the abstract)	Yes	Partial – execution-based, not full exploit confirmation	5 languages, incl. Python, C, Java	31 CWE types	Uses outcome-driven execution checks to confirm a flaw is triggerable, but not that it forms a full attacker-controlled exploit path.
Security Degradation in Iterative AI Code Generation (Shukla et al.)	1 model: GPT-4o	Yes	No	C, Java	Not organized by CWE	Shows vulnerabilities increase over iterative refinement rounds, but never confirms whether those added vulnerabilities are actually exploitable.
Human-Written vs. AI-Generated Code (Cotroneo et al.)	3 models: ChatGPT, DeepSeek-Coder, Qwen-Coder	Yes	No	Python, Java	Not organized by CWE	Large-scale comparison of vulnerability rates between human and AI code, but never tests whether flagged vulnerabilities are exploitable.
AI Code Assistants & Security API Usage (Mousavi et al.)	1 tool: GitHub Copilot	Partial (developer awareness measured, not automated detection)	No	Java	Not organized by CWE; focused on security-API misuse patterns	Human study showing developers don't reliably catch vulnerabilities Copilot introduces; no automated detection or exploit confirmation.

Summary. Of the 19 papers and benchmarks reviewed, only two (Broken by Default, PoCGen) confirm that a vulnerability is actually exploitable at scale, and both answer a different question than this thesis: one proves memory-corruption bugs in C/C++ with a mathematical method, the other exploits already-known CVEs rather than vulnerabilities an AI model freshly introduces. Two more (Yoo & Kim, CWEval) confirm exploitability only partially – for two hand-picked examples, or via execution checks that stop short of a full attacker-controlled exploit path. Three entries (SecurityEval Dataset, PRIMEVUL, Multi-Language SVD) are datasets or detection-only benchmarks where exploit confirmation does not apply. Every remaining paper – twelve in total, spanning Copilot, ChatGPT, and other model families across Java, Python, C/C++, and JavaScript – detects vulnerabilities but never confirms whether the flagged code can actually be exploited. No paper reviewed functionally confirms exploitability for SQL Injection, Command Injection, and Path Traversal specifically, across multiple AI models, in Python, Java, or JavaScript. That is the gap this thesis will address.